1. MCP-SOAR: AI-Native Behavioral Threat Detection System

Report Period: Week ending May 2026 **Project:** Graduation Thesis — AI-Powered SOAR via Model Context Protocol

Status: **Core pipeline completed and end-to-end verified**

1.1 Core Concept and Motivation

Traditional SOAR systems only react when a detection tool fires an alert. This system monitors **every user action** — including normal browsing — and lets AI analyze the full behavioral chain over time.

Key insight: Each of the following requests is harmless in isolation.

`GET /robots.txt` → normal `GET /.env` → normal `GET /admin` → normal
`POST /login fail` → normal `GET /internal-api` → normal

The chain: reconnaissance pattern — detected by AI, missed by Wazuh.

1.2 Why MCP (Model Context Protocol)?

- Each server exposes **tools with docstrings** — the AI host discovers capabilities automatically.
- The orchestrator makes only **4 MCP calls** per alert: `collect` → `translate` → `correlate` → `update_context`.
- Any server can be replaced or scaled independently — orchestrator code never changes.
- Tool inputs/outputs are plain `dict` — no manual JSON serialization between layers.

1.3 System Architecture

1.3.1 Component Overview

Component	Responsibility
`collector_server`	Aggregates logs from 7 sources via MCP tools
`translator_server`	Classifies ALL user actions — attacks and normal behavior
`correlator_server`	Session queue, risk scoring, flush logic, context memory
`reasoning_server`	AI analysis with dual prompt mode
`response_server`	Generates and executes SOAR action plans
`orchestrator.py`	Coordinates 4 MCP calls, handles branch A / branch B
`mcp_client.py`	MCP SSE transport wrapper
`wazuh_client.py`	Async Wazuh Indexer poller (Trigger 1)
`main.py`	Dual trigger entry point

1.3.2 Log Sources

Source	Path in container	Detects
web	`/shared_logs/access.log`	HTTP requests, URL patterns
container	`/shared_logs/error.log`	Application errors
auth	`/var/log/auth.log`	SSH brute force, invalid users
db	docker logs dvwa-mysql	Database query anomalies
audit	`/var/log/audit/audit.log`	Privilege escalation, sudo
firewall	`/var/log/ufw.log`	Port scans, blocked connections
wazuh	Wazuh Indexer API	SIEM alerts (Trigger 1 only)

1.4 Dual Trigger Design

Two independent triggers run simultaneously inside `main.py`:

Trigger	Interval	Sources used
Trigger 1: Wazuh alert poll	10s	`fetch_all_sources()` — 7 sources
Trigger 2: Web log poll via MCP	30s	6 sources, NO Wazuh

1.4.1 Branch Logic

Branch A (`has_wazuh_alert = True` — confirmed attack):

1. Execute immediate active response without waiting for admin
2. AI generates full incident report and patch suggestion

Branch B (`has_wazuh_alert = False` — behavioral monitoring):

1. AI analyzes full behavioral chain including normal actions
2. **Suspicious**: response plan generated, admin decision required
3. **Normal**: forensic snapshot stored for 30 minutes, then deleted

1.5 Queue Formula and Context Memory

1.5.1 Flush Conditions

Condition	Behavior
Queue contains Wazuh alert	Immediate flush
≥ 2 different attack types	Immediate flush
Queue size ≥ 30 events	Threshold flush
Risk score ≥ 40	Threshold flush
30-minute timeout	Always — even quiet sessions

1.5.2 Formula

$$\text{payload\_to\_AI}(n) \;=\; \text{context\_memory}\big(AI_1, \ldots, AI_{n-1}\big) \;+\; \text{behavior\_queue}(n)$$

Every AI verdict is stored in `context_memory`. The next flush carries all prior verdicts plus new behaviors — AI builds understanding of a session over time.

1.5.3 Real Example Captured During Testing

The following timeline was recorded from a live DVWA session. The tester browsed normally across multiple queue cycles, then launched a real attack.

Time	Queue Contents	AI Verdict	Risk
22:31	22 normal + 2 LFI	probe_detected	32.8
22:36	20 normal + 4 LFI (saw Q1)	probe_detected	63.4
22:41	saw Q1+Q2	probe_detected	63.3
22:46	saw Q1–Q3	probe_detected	63.1
22:51	saw Q1–Q4	probe_detected	62.9
22:56	saw Q1–Q5, no Wazuh alert	**is_real_attack=TRUE, severity=HIGH, confidence=0.95**	62.7
22:57	Wazuh fires	attack_confirmed, active response executed	100.0

Queue 6 (22:56) is the key proof: AI escalated verdict to `is_real_attack = TRUE` with `confidence = 0.95` based purely on accumulated behavioral context — **one full minute before Wazuh fired any alert**. This demonstrates the system's ability to detect attacks that signature-based tools miss.

1.6 Behavior Classification

The translator labels all user actions. This is essential for AI to see full behavioral context.

Category	Actions
Attack indicators	`authentication_bypass_attempt`, `local_file_inclusion_attempt`, `command_injection_attempt`, `cross_site_scripting_attempt`, `ssrf_attempt`, `webshell_execution`, `reverse_shell_attempt`, `brute_force_attempt`, `privilege_escalation_attempt`, `port_scan_detected`, `wazuh_critical_alert`
Normal actions	`page_browse`, `api_call`, `form_submit`, `login_page_visit`, `logout`, `search_query`, `profile_access`, `settings_access`
Filtered (noise)	Static assets: `.css`, `.js`, `.png`, `.ico`, `.woff`, `.svg`

1.7 AI Analysis Modes

Mode `behavioral_analysis` (Branch B — no Wazuh): looks for reconnaissance patterns, automated scanning, probing sequences across the full chain of normal and suspicious actions.

Mode `incident_report` (Branch A — Wazuh confirmed): summarizes attack chain, assesses impact, and suggests a specific code or configuration patch.

Provider	Free	Model
Groq	✓ 14,400/day	`llama-3.3-70b-versatile`
Gemini	✓ rate limited	`gemini-2.0-flash`
Claude	×	`claude-sonnet-4-20250514`
OpenAI	×	`gpt-4o-mini`

1.8 Volume Mount Setup

The collector reads web and container logs through a shared Docker volume. DVWA must write Apache logs to the correct host path.

Step 1 — Create shared log directory:

```
mkdir -p /home/kali/my-lab/infra/volumes/shared_logs/dvwa
```

Step 2 — Mount DVWA Apache logs into this path:

```
volumes:
  - /home/kali/my-lab/infra/volumes/shared_logs/dvwa:/var/log/apache2
```

Step 3 — Verify required files exist:

```
ls /home/kali/my-lab/infra/volumes/shared_logs/dvwa/
# Required: access.log  error.log
```

Step 4 — Collector mounts this read-only (already configured):

```
collector_server:
  volumes:
    - /home/kali/my-lab/infra/volumes/shared_logs/dvwa:/shared_logs:ro
```

Inside the collector: `/shared_logs/access.log` (web) and `/shared_logs/error.log` (container errors).

1.9 Completed Implementation Checklist

	Component / Feature
✓	**Collector MCP Server** — 7 log sources, `fetch_all_sources` fan-out, IP-level cache, sanitized grep
✓	**Translator MCP Server** — 15+ attack patterns, normal behavior classification, compiled regex, hex/URL decode bypass
✓	**Correlator MCP Server** — session queue, exponential risk decay ($e^{-t/300}$), 5 flush conditions, `has_wazuh_alert` flag, context memory, 30-min forensic retention
✓	**Reasoning MCP Server** — dual prompt mode, supports Groq / Gemini / Claude / OpenAI via environment variable
✓	**Response MCP Server** — tiered actions by severity, `execute_active_response` for Branch A
✓	**Orchestrator** — 4 MCP calls per alert, dual branch, shared `_run_pipeline`
✓	**Dual Trigger main.py** — Wazuh poll (10s) + web log poll (30s) via collector MCP
✓	**Context memory formula** — queue$_n = AI_{n-1}$ + behavior$_n$ verified with real attack timeline
✓	**End-to-end pipeline** — tested with real DVWA; behavioral detection confirmed before Wazuh fired
✓	**Test suite** — 9 test cases: unit detection, noise filter, risk scoring, flush conditions, schema compatibility, MCP surface

1.10 Known Limitations

Area	Current State	Upgrade Path
Session store	JSON file	Redis Streams / SQLite WAL
Log collection	grep subprocess	Filebeat / Elastic ingest
AI confidence	Fixed heuristics	Weighted scoring / ML layer
Process monitoring	Not implemented	`/proc` + Docker API MCP server
Multi-instance	Single node	Redis-backed shared session store